ASSIGNMENT 1 – CO1 | TASK 9

Implement and Test Fare Logic

Scenario: Namma Metro Smart-Card Fare System

Objective / explanation

This task tests normal fares, peak boundaries and invalid input. The fare table is evaluated with a set of test cases so that new tests can be added as rows.

C++ source code

```
#include &lt;iostream&gt;
#include &lt;vector&gt;
using namespace std;

long long fare(int zone, int hour) {
    if (zone &lt; 0 || zone &gt; 2 || hour &lt; 0 || hour &gt; 23) return -1;
    long long base = (zone + 1) * 1000LL;
    bool peak = (hour &gt;= 8 &amp;&amp; hour &lt; 10) || (hour &gt;= 17 &amp;&amp; hour &lt; 20);
    return peak ? base + base * 20 / 100 : base;
}
int main() {
    struct Test { int zone, hour; long long expected; };
    vector&lt;Test&gt; tests = {{0,12,1000},{1,12,2000},{2,8,3600},
                          {2,10,3000},{2,20,3000},{0,17,1200}};
    int passed = 0;
    for (const auto&amp; t : tests) {
        long long got = fare(t.zone, t.hour);
        bool ok = got == t.expected;
        cout &lt;&lt; "zone=" &lt;&lt; t.zone &lt;&lt; " hour=" &lt;&lt; t.hour
             &lt;&lt; " expected=" &lt;&lt; t.expected &lt;&lt; " actual=" &lt;&lt; got
             &lt;&lt; " " &lt;&lt; (ok ? "PASS" : "FAIL") &lt;&lt; '\n';
        passed += ok;
    }
    cout &lt;&lt; "Passed: " &lt;&lt; passed &lt;&lt; "/" &lt;&lt; tests.size() &lt;&lt; '\n';
}
```

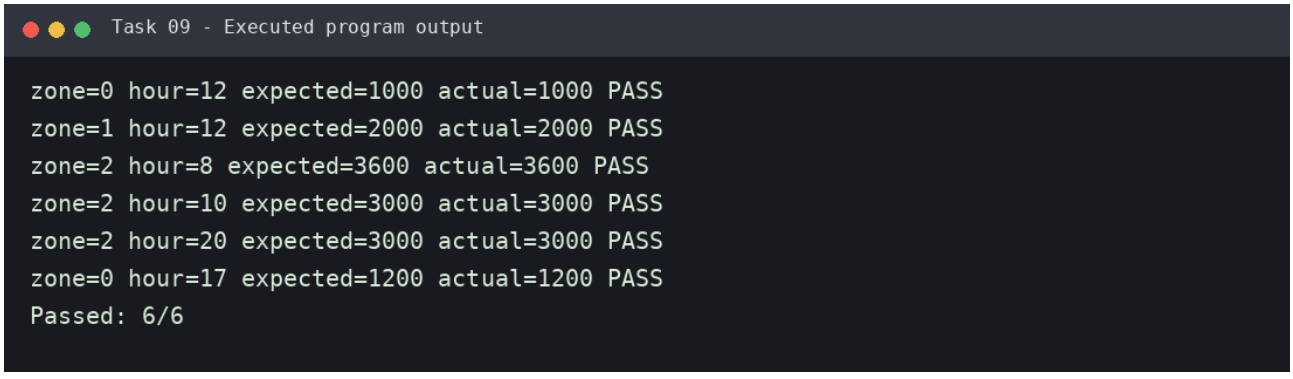
Compilation

```
g++ -std=c++17 -Wall -Wextra -pedantic main.cpp -o program
```

Actual captured program output

```
zone=0 hour=12 expected=1000 actual=1000 PASS
zone=1 hour=12 expected=2000 actual=2000 PASS
zone=2 hour=8 expected=3600 actual=3600 PASS
zone=2 hour=10 expected=3000 actual=3000 PASS
zone=2 hour=20 expected=3000 actual=3000 PASS
zone=0 hour=17 expected=1200 actual=1200 PASS
Passed: 6/6
```

Output screenshot



Analysis / complexity

A table-driven test loop takes O(t) time and O(t) space for t cases. The boundary rule is 08:00–09:59 and 17:00–19:59. Include invalid zone/hour tests in a full implementation.

Evidence note: Output shown is captured by executing this source in the generation environment. Recompile and run on your own system, and use your own screenshots if your instructor requires evidence from your computer.